

2021 Paper

Question 1

(a)

Consider a transistor amplifier circuit. To counter the effects of small fluctuations in the supply voltage, a small resistor is introduced connecting the source to ground. Explain how this works.

(b)

i. You need to design a system for the transmission of surround audio channels (7+1). The maximum frequency of each channel is 22 KHz, however, you need to oversample the signal at twice the minimum sampling rate. Calculate the total bandwidth required to transmit the 7+1 channels, if they are time division multiplexed, quantised at 16 bits per sample and use a 16-QAM modulation with carrier frequency centred at 100 MHz.
ii. Then assume you need to transmit 10 such multi-channel surround systems. The 10 systems are multiplexed using frequency division multiplexing, with guard bands of 60 KHz. However, your maximum available bandwidth is 20 MHz. What modulation format do you need to use to fit the 10 systems in the given 20 MHz bandwidth?
iii. Assume you need to reduce the overall bandwidth to 10 MHz, but you cannot further increase (i.e., with respect to point ii) above) the number of bits per sample in the modulation format. Which other action could you carry out without noticeably compromising the signal quality? Justify your answer showing your calculations.

Attempt:

(a) The introduction of a small resistor from the source to ground in a transistor amplifier circuit is often referred to as “emitter degeneration” or “emitter resistor feedback”. This

technique helps counter the effects of small fluctuations in the supply voltage and provide several benefits to the amplifier's performance. This is how it work:

1. Negative Feedback:
    a. The emitter resistor introduces negative feedback into the circuit
    b. Negative feedback tends to stabilise the amplifier and reduce its sensitivity to variations in different parameters.
2. Decoupling from Supply Voltage Fluctuations:
    a. Small Fluctuations in the supply voltage can cause changes in the operating point of the transistor.
    b. The emitter resistor helps decouple the emitter current from these variations by providing an alternative path for the current to flow through
3. Current Stability:
    a. The emitter resistor R_e, introduces a voltage drop proportional to the emitter current I_e, according the Ohm's law (V = IR)
    b. As the supply voltage fluctuates, the emitter resistor adjusts the emitter current, helping to maintain a more stable bias point.
4. Reduce Sensitivity to Transistor Parameters:
    a. The emitter degeneration technique reduces the sensitivity of the amplifier to changes in transistor parameters, such as the transistor's beta value.
    b. Variations in beta can affect the gain of the amplifier, but the negative feedback provided by the emitter resistor help mitigate this effect.
5. Improve Linearity:
    a. Negative feedback improves linearity of the amplifier by reducing distortion.
    b. By controlling the operating point of the transistor, the emitter resistor helps to maintain a more linear relationship between inout and output signal.
6. Thermal Stability:
    a. The emitter resistor also contributes to thermal stability.
    b. As the temperature changes, the emitter resistor helps stabilise the bias point, reducing the impact of temperature variations on the amplifier's performance.

In summary, the introduction of an emitter resistor in a transistor amplifier provide negative feedback, stabilising the amplifier against small fluctuations in the supply voltage and other environmental factors. This technique improves the overall performance of the amplifier by enhancing stability, linearity and thermal characteristics.

(b)

i

$$\text{Bit Rate} = 2 * \text{Bandwidth} * log_2 L$$
$$\text{Bit Rate} = \text{number of bits per sample } * sample\ frequency\ * \text{number of channel}$$

16 QAM → 16 = 2^n , n = 4.

$$B_{mod} = (1 + d) * S$$
$$S = \frac{R}{n}$$
$$R = \text{number of bits per sample } * sample\ frequency\ * \text{number of channel}$$

channels = 8

max f of each channel = f_max = 22KHz

oversample the signal at twice the minimum sampling rate.

16 bits per sample

carrier f = 100MHz

$$f_{sample} = 2 * f_{max}$$
$$f_{sample} = 2 * 22kHz = 44kHz$$

Need to oversample:

$$44kHz * 2 = 88kHz$$

Total Bandwidth:

$$Taking\ d = 0$$
$$B_{mod} = \frac{R}{log_2 16}$$
$$R = 16 * 88kHz * 8$$
$$B_{mod} = \frac{11264000}{4} = 2816000Hz$$

ii

need 10 of the systems.

frequency division multiplexing

guard bands = 60KHz

max available bandwidth = 20KHz

what modulation format needs to fit 10 system within the given 20 MHz bandwidth

Bandwidth of guard bands

bruh i can't.

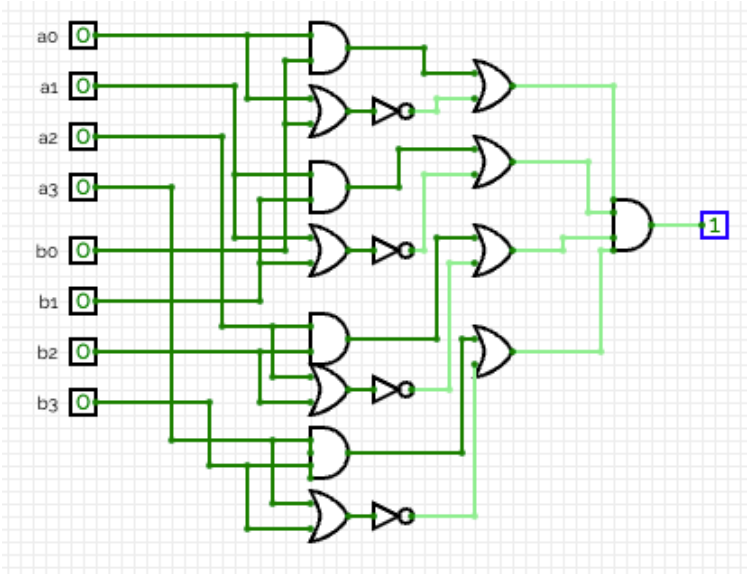
yes i can

If each guard band is 60K Hz wide, and there is one between each system, that means that there is 9 total guard bands equalling combined bandwidth of 54 k Hz. Therefore we have to fit 10 systems within 20000 - 54 = 19,946 M Hz. Therefore the bandwidth of one system cannot exceed 199946 M Hz,

2.816 M Hz is far too large. Let's try increasing the 16-QAM modulation to 32-QAM. This yields a bandwidth of 2252000

Question 2

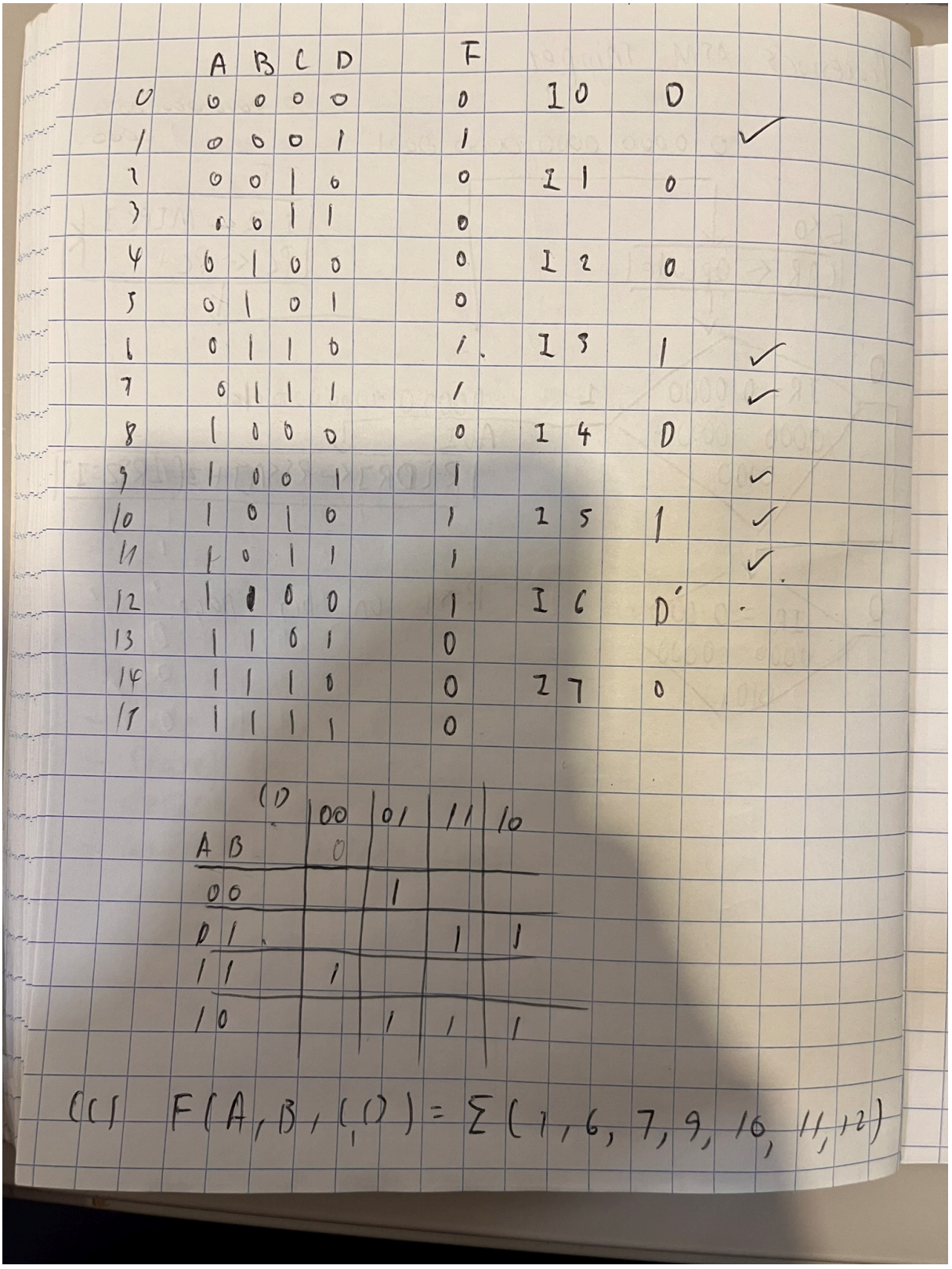
1. (a) Design a combinational circuit that compares two 4-bit numbers to check if they are equal. The circuit output is equal to 1 if the two numbers are equal and 0 otherwise. Provide a Boolean algebra equation of your solution and a circuit diagram.



F = AB+!A!B

(b) An 8 x 1 multiplexer has inputs A, B, and C connected to the selection inputs S2, S1 and S0, respectively. The data inputs I0 through I7 are as follows:
I1=I2=I7=0;I3=I5=1; I0=I4=D; and I6=D'. Draw a K map of the function
the multiplexer implements.

(c) Using sigma notation, list the min-terms of the function the multiplexer implements.



Question 3

Design an Algorithm State Machines (ASM) for a multiplication instruction that can be implemented on a modified version of the processor that you designed and simulated as part

of your coursework for the "CSU22022 - Computer Architecture I" module. The processor architecture is shown in *Figure 5*. The ASM should implement a Two's Complement Multiplication. Product (32 bit) = Multiplicand (16 bit) x Multiplier (16 bit).

- You should provide an ASM chart for the two's complement multiplication and a written description for the various states and the state transitions of the ASM. This should include all register transfer operations that need to be executed.
- You should also populate the first 15 consecutive Control Memory Addresses that implement the first 15 states of your ASM. Therefore, show how your ASM design is implemented on the modified architecture in *Figure 5*. A Control Memory template is shown in *Figure 6*.

Please assume the following:
The Two's Complement Multiplication Instruction has the following format *Figure 1*:

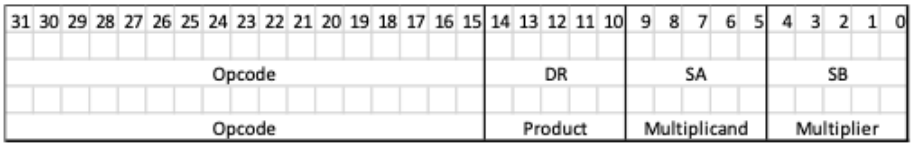


Figure 1 Two's Complement Multiplication Instruction

DR – is the address of the register that will receive the output of your two's complement multiplication. The Product should be in a 32 bit two's complement format.

SA - is the address of the register that holds the 16 bit Multiplicand in a two's complement format. The 16 bit Multiplicand is sign extended to 32 bit.

SB - is the address of the register that holds the 16 bit Multiplier in a two's complement format. The 16 bit Multiplicand is sign extended to 32 bit.

The Multiplicand and the Multiplier can be positive or negative. Therefore, your ASM design needs to be able to compute the four combinations shown in *Figure 2*.

Multiplicand	Multiplier
negative	positive
negative	negative
positive	positive
positive	negative

The processor in *Figure 5* implements the functions shown in *Figure 3*. The F = srB and F = slB functions perform a single bit shift. They insert a '0' in the vacated MSB or LSB position. Furthermore, these shift operations will not set the carry flag.

Table 1: FS code definition

FS	MF Select	G Select	H Select	Micro-operation
00000	0	0000	00	$F = A$
00001	0	0001	00	$F = A + 1$
00010	0	0010	00	$F = A + B$
00011	0	0011	00	$F = A + B + 1$
00100	0	0100	01	$F = A + \bar{B}$
00101	0	0101	01	$F = A + \bar{B} + 1$
00110	0	0110	01	$F = A - 1$
00111	0	0111	01	$F = A$
01000	0	1000	00	$F = A \wedge B$
01010	0	1010	10	$F = A \vee B$
01100	0	1100	10	$F = A \oplus B$
01110	0	1110	10	$F = \bar{A}$
10000	1	0000	00	$F = B$
10100	1	0100	01	$F = srB$
11000	1	1000	10	$F = slB$

figure 3: Function code

The modified processor show in *Figure 5* has changed the T-signals to 3 bit TD, 3 bit TA, and 3 bit TB signals. This provides you with 7 “Temp Registers”. *Figure 4* shows how you can access these registers. If TD, TA, or TB are set to ‘000’ DR, SA, and SB will select one of the 32 registers in the register-file. Settings other than ‘000’ will give you access to the 7 “Temp Registers”.

TD				TA				TB			
0	0	0	DR selected Register	0	0	0	SA selected Register	0	0	0	SB selected Register
0	0	1	T0	0	0	1	T0	0	0	1	T0
0	1	0	T1	0	1	0	T1	0	1	0	T1
0	1	1	T2	0	1	1	T2	0	1	1	T2
1	0	0	T3	1	0	0	T3	1	0	0	T3
1	0	1	T4	1	0	1	T4	1	0	1	T4
1	1	0	T5	1	1	0	T5	1	1	0	T5
1	1	1	T6	1	1	1	T6	1	1	1	T6

Figure 4 TD-DR, TA-SA, and TB-SB

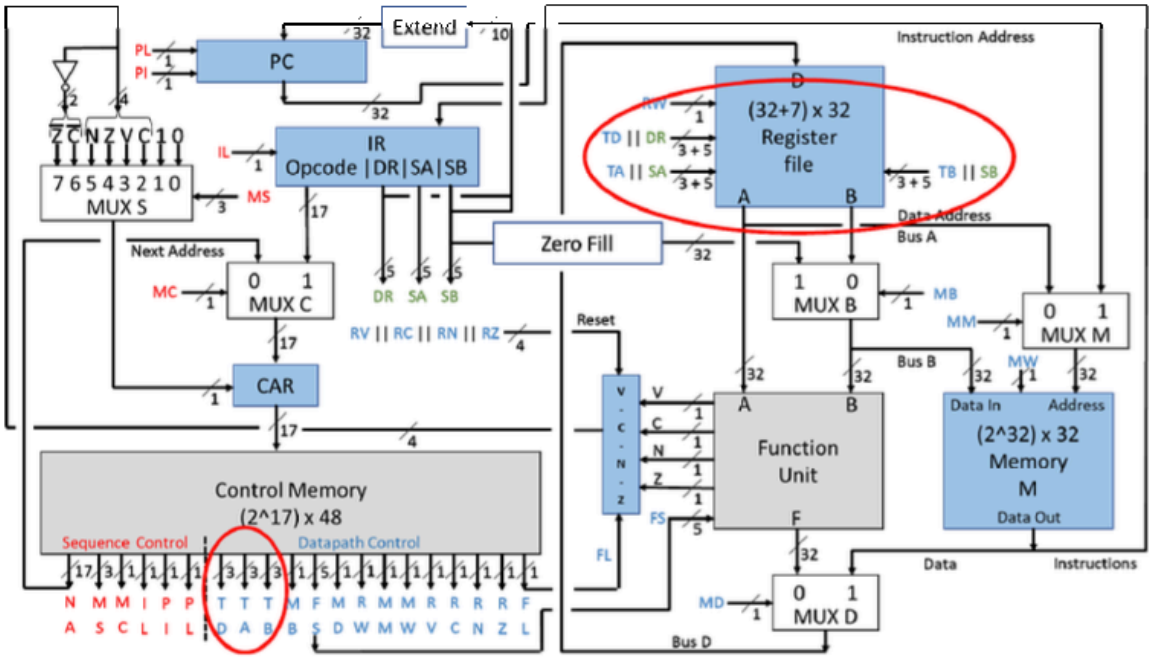


Figure 5 Processor with 3 bit TD, 3 bit TA, and 3 bit TB




```
architecture Behavioral of control_memory is
type mem_array is array(0 to 255) of std_logic_vector(47 downto 0);

begin

memory_m: process(IN_CAR)
        variable control_mem : mem_array:=(
-- |47           31|3028|27|26|25|24|2321|2018|1715|14|13  9|8|7|6|5|4|3|2|1|0|
-- | Next Address  | MS | M| I| P| P| TD | TA | TB | M|  FS |M|R|M|M|R|R|R|R|F|
-- |               |    | C| L| I| L|    |    |    | B|     |D|W|M|W|V|C|N|Z|L|
  "00000000000000000 000  0  0  0  0  000  000  000  0 00000 0 0 0 0 0 0 0 0 0",-- 00
  "00000000000000000 000  0  0  0  0  000  000  000  0 00000 0 0 0 0 0 0 0 0 0",-- 01
  "00000000000000000 000  0  0  0  0  000  000  000  0 00000 0 0 0 0 0 0 0 0 0",-- 02
  "00000000000000000 000  0  0  0  0  000  000  000  0 00000 0 0 0 0 0 0 0 0 0",-- 03
  "00000000000000000 000  0  0  0  0  000  000  000  0 00000 0 0 0 0 0 0 0 0 0",-- 04
  "00000000000000000 000  0  0  0  0  000  000  000  0 00000 0 0 0 0 0 0 0 0 0",-- 05
  "00000000000000000 000  0  0  0  0  000  000  000  0 00000 0 0 0 0 0 0 0 0 0",-- 06
  "00000000000000000 000  0  0  0  0  000  000  000  0 00000 0 0 0 0 0 0 0 0 0",-- 07
```

Figure 6 Control Memory with 3 bit TD, 3 bit TA, and 3 bit TB

Attempt:

1. State 0 (Initialisation):
    a. Set Control signal to initialise the operation
    b. Set the set to the next state

2. State 1 (Load Multiplicand and Multiplier)
    a. Load Multiplicand from memory (Register Addressed by SA) into a register
    b. Load Multiplier from memory(Register Addressed by SB) into another register
    c. Sign-extend both Multiplicand and Multiplier to 32 bits.
    d. Set the state to next state
3. State 2 (Initialise Product and Counter)
    a. Initialise the Product register to 0
    b. initialise a counter to the number of bits in the Multiplier.
    c. set the state to next state
4. State 3(Multiply Loop)
    a. If the least significant bit if the multiplier is 1, add the extended Multiplicand to the Product.
    b. Right shift the Product
    c. Right shift the Multiplier
    d. Decrease the counter
    e. If the counter is not zero, go back to State 3; otherwise, go to the next state.
5. State 4(Check Overflow)
    a. Check for overflow (discard overflow if any)
    b. Set the state to the next one
6. State 5 (Store Result)
    a. Store the result into memory location addressed by DR
    b. Set state to the next one
7. State 6 (Stop)
    a. Halt the processor or proceed to net instruction